

Übung 2: Daten einlesen und mit dem tidyverse aufbereiten

Zugehöriger Foliensatz: Teil II (Daten einlesen und mit dem tidyverse verarbeiten)

Arbeitsverzeichnis und Daten

- RStudio-Projekt: `projekt/` (Datei `vgr-r-schulung.Rproj` öffnen).
- Datenzugriff immer relativ als `data/...`
- Datensatz: `data/lieferung_2024q4.csv`. Tidy/long mit den Spalten `jahr`, `quartal`, `bereich_code`, `bereich_name`, `wert` (Mio. EUR, jeweilige Preise). 280 Zeilen, 10 Wirtschaftsbereiche, Jahre 2018 bis 2024.

Datei für diese Übung

- Starterdatei: `projekt/R/lab2_einlesen_tidyverse.R` (mit # TODO-Lücken).

Sollwerte zur Selbstkontrolle

Prüfen Sie Ihre Ergebnisse gegen diese Werte:

- Eingelesene Zeilen: **280**, Spalten: **5**.
- Zahl der Wirtschaftsbereiche (`distinct`): **10**.
- **Gesamt-BWS 2024 (Summe aller vier Quartale, alle Bereiche): 3.827.201,0 Mio. EUR.**
- BWS-Summe nur 2024Q4: rund 996.430 Mio. EUR.
- Größter Bereich in 2024Q4: `BE` (Produzierendes Gewerbe ohne Baugewerbe).

Schritt 1: Pakete laden und CSV einlesen

`read_csv()` liefert ein `tibble`, meldet die erkannten Spaltentypen und lässt Text als Text. Geben Sie die Typen mit `col_types` vor, dann fällt eine abweichend formatierte Lieferung sofort auf.

```
library(tidyverse)

lieferung <- NULL # TODO: mit read_csv() einlesen und col_types festlegen
```

Aufgabe: Lesen Sie die Datei ein und legen Sie die Typen wie oben fest.

Hinweis: Lassen Sie `col_types` zunächst weg und betrachten Sie den automatischen Typ-Report. Mit `spec(lieferung)` sehen Sie die geratene Spezifikation als Vorlage.

Schritt 2: Erster Blick mit `glimpse()`

```
# TODO: Struktur von lieferung mit glimpse() prüfen
```

Aufgabe: Geben Sie `glimpse(lieferung)` aus und prüfen Sie: Stehen `jahr` und `quartal` als `<int>` und `wert` als `<dbl>` da?

Hinweis: `glimpse()` ist gerade bei vielen Spalten übersichtlicher als `head()`, weil es die Spalten untereinander zeigt.

Schritt 3: Filtern mit `filter()`, `%in%` und `between()`

```
q4_2024 <- NULL # TODO
zeitraum <- NULL # TODO
```

Aufgabe: Erzeugen Sie `q4_2024` (2024, Quartal 4) und `zeitraum` (Jahre 2022 bis 2024).

Hinweis: Mehrere Bedingungen mit Komma sind ein logisches UND. Auf Gleichheit prüfen Sie mit `==`, nicht mit `=`.

Schritt 4: Neue Kennzahlen mit `mutate()` und `lag()`

```
anteile <- NULL # TODO
veraenderung <- NULL # TODO
```

Aufgabe: Berechnen Sie `anteile` (Anteil am Quartalstotal) und `veraenderung` (Vorquartalswert mit `lag()` plus relative Veränderung).

Hinweis: `lag()` greift auf den vorherigen Wert in der sortierten Reihenfolge zu. Sortieren Sie deshalb mit `arrange()`, bevor Sie `lag()` je Bereich anwenden.

Schritt 5: Aggregieren mit `group_by()` und `summarise()`

```
bws_je_jahr <- NULL # TODO
mittel_je_bereich <- NULL # TODO
```

Aufgabe: Berechnen Sie `bws_je_jahr` und `mittel_je_bereich`.

Hinweis: Setzen Sie `.groups = "drop"` (oder `ungroup()`), sonst rechnen spätere Schritte unbeabsichtigt je Gruppe weiter.

Schritt 6: Sortieren und Top-Bereiche

```
top_bereiche <- NULL # TODO
```

Aufgabe: Bestimmen Sie die drei größten Bereiche in 2024Q4.

Hinweis: `desc()` sortiert absteigend. `slice_head(n = 3)` behält die ersten drei Zeilen.

Schritt 7: Zählen mit `count()` und `distinct()`

```
zeilen_je_jahr <- NULL # TODO
bereiche <- NULL # TODO
```

Aufgabe: Zählen Sie die Zeilen je Jahr und listen Sie die eindeutigen Bereiche.

Hinweis: `count(jahr)` ist die Kurzform für `group_by(jahr) |> summarise(n = n())`. Je Jahr sollten genau 40 Zeilen stehen (10 Bereiche × 4 Quartale).

Schritt 8: Format wechseln mit `pivot_wider()`

```
breit <- NULL # TODO
```

Aufgabe: Erzeugen Sie `breit` mit den Bereichen in Zeilen und je Quartal (2018Q1 ...) eine Spalte.

Hinweis: `pivot_wider()` braucht eindeutige Schlüssel je Zeile. Hier ist das die Kombination aus `bereich_code` und `zeit`.

Schritt 9: Kennzahl ausgeben

```
gesamt_bws_2024 <- NULL # TODO  
# TODO: Kennzahl im deutschen Zahlenformat ausgeben
```

Aufgabe: Geben Sie die Gesamt-BWS 2024 aus und vergleichen Sie mit dem Sollwert oben (3.827.201,0).

Hinweis: `pull()` zieht eine einzelne Spalte als Vektor heraus. `format(..., big.mark = ".", decimal.mark = ",")` formatiert die Zahl im deutschen Format.